

LANG SON OJ

TÀI LIỆU HƯỚNG DẪN SỬ DỤNG HỆ THỐNG

Cẩm nang toàn diện dành cho Học sinh, Thí sinh tham gia luyện tập lập trình thi đấu và Giáo viên / Ban quản trị vận hành tổ chức bài tập, tạo testcase tự động.

Chấm bài tự động siêu tốc

Tích hợp Piston Judge Engine, hỗ trợ biên dịch và thực thi chuẩn xác C++ (GCC), Python 3 và Java (OpenJDK).

Mở khóa Hidden Testcase

Sau khi nộp bài, thí sinh có thể mở khóa chi tiết các test ẩn để debug, đối chiếu input, output và lý do sai.

Tạo bài nhanh từ File Mẫu

Admin có thể upload file mẫu `sample.txt` để tạo bài tập kèm toàn bộ testcase tự động chỉ với 1 click.

Hỗ trợ toán học LaTeX & Markdown

Đề bài hiển thị chuẩn công thức toán học KaTeX, Markdown phong phú, giao diện Dark Theme tiêu chuẩn hiện đại.

1. GIỚI THIỆU TỔNG QUAN HỆ THỐNG

LANG SON OJ (Lạng Sơn Online Judge) là nền tảng chấm bài lập trình trực tuyến hiện đại được thiết kế chuyên biệt cho việc ôn luyện, bồi dưỡng học sinh giỏi và tổ chức các kỳ thi tin học thi đấu (Competitive Programming). Hệ thống tập trung vào trải nghiệm mượt mà, tốc độ phản hồi nhanh, hỗ trợ đầy đủ các ngôn ngữ lập trình phổ biến và cơ chế tạo đề thi cực kỳ tiện lợi cho giáo viên.

Kiến trúc kỹ thuật cốt lõi:

- **Giao diện & Nền tảng web:** Next.js 14 App Router, React 18, Tailwind CSS, TypeScript.
- **Trình soạn thảo mã nguồn:** Monaco Editor / CodeMirror 6 chuyên dụng cho lập trình thi đấu, tích hợp đầy đủ phím tắt, đổi theme, gợi ý code và thực lệ thông minh.
- **Công cụ chấm bài (Judge Engine):** Piston API v2 Engine, thực thi mã nguồn trong môi trường sandbox cô lập an toàn, đo lường thời gian chạy (ms) và dung lượng bộ nhớ (KB) chuẩn xác.
- **Cơ sở dữ liệu:** PostgreSQL trên nền tảng Supabase Cloud với kết nối Connection Pooling tốc độ cao.

✦ **Lưu ý bản quyền:** Hệ thống được thiết kế và xây dựng bởi tác giả **NgHuyHoang**. Mọi tính năng cốt lõi được tối ưu hóa cho nhu cầu luyện tập thi đấu học sinh giỏi các cấp.

2. HƯỚNG DẪN DÀNH CHO HỌC SINH / THÍ SINH

2.1. Tra cứu và lựa chọn bài tập

Khi truy cập mục "**Danh sách bài tập**" (/problems), bạn sẽ thấy danh mục tất cả bài tập có trên hệ thống với các thông tin tóm tắt:

- **Mức độ khó:** Được phân loại rõ ràng bằng nhãn màu: **Dễ (EASY)** , **Trung bình (MEDIUM)** , **Khó (HARD)** .
- **Thẻ chủ đề (Tags):** Lọc nhanh theo dạng bài như **Quy hoạch động** , **Đồ thị** , **Toán học** , **Hai con trỏ** ,...
- **Thanh tìm kiếm:** Nhập từ khóa tên bài hoặc slug để tìm bài nhanh chóng.
- **Thống kê:** Số lượng testcase và số lượt nộp bài đã được ghi nhận.

2.2. Giao diện làm bài chi tiết

Khi chọn một bài tập, màn hình được chia thành hai phần trực quan:

- **Cột bên trái - Nội dung đề bài:**
 - Mô tả bài toán chi tiết, có định dạng công thức toán học LaTeX (ví dụ: $O(N \log N)$, $\$1 \le N \le 10^5\$$).
 - Quy cách định dạng Input và Output.
 - Ràng buộc và giới hạn: **Giới hạn thời gian (Time Limit)** (ví dụ: 1.0 giây) và **Giới hạn bộ nhớ (Memory Limit)** (ví dụ: 256 MB).
 - Ví dụ mẫu (Sample Input / Sample Output) kèm giải thích chi tiết.
- **Cột bên phải - Trình soạn thảo mã nguồn (Code Editor):**
 - **Chọn ngôn ngữ:** C++ (GCC), Python 3, Java (OpenJDK). Mỗi ngôn ngữ đều có sẵn mã khung (template) chuẩn.
 - **Tùy chỉnh cá nhân:** Đổi theme (Dark / Light), cỡ chữ (12px - 18px), nút Reset Code để quay về template ban đầu.

2.3. Chạy thử nghiệm với dữ liệu tự chọn (Run Code)

Trước khi nộp bài chính thức, thí sinh có thể kiểm tra thuật toán bằng cách:

1. Bật tùy chọn "**Dữ liệu đầu vào tùy chỉnh (Custom Input / Stdin)**" ở tab bên dưới editor.
2. Nhập dữ liệu test tự tạo vào ô nhập.
3. Nhấn nút "**Chạy thử**" (nút màu xám/xanh có biểu tượng Play).
4. Hệ thống sẽ biên dịch và thực thi ngay lập tức, trả về Output thực tế, thời gian chạy và thông báo lỗi (nếu có) mà không lưu vào lịch sử nộp bài.

2.4. Nộp bài chính thức (Submit Code)

Khi đã tự tin với bài làm, thí sinh bấm nút "**Nộp bài**" (**Submit**) màu xanh lá cây:

1. Hệ thống gửi mã nguồn lên Judge Engine để chấm lần lượt qua **tất cả các testcase** (bao gồm cả test mẫu và test ẩn bí mật).
2. Tiến trình chấm được cập nhật theo thời gian thực (Trạng thái `Processing`).
3. Khi hoàn tất, bảng kết quả tổng quát sẽ hiển thị: Trạng thái (AC, WA,...), số test vượt qua (ví dụ: `10/10 PASS`), thời gian thực thi tối đa và dung lượng bộ nhớ.

2.5. Tính năng xem chi tiết & Mở khóa Hidden Tests

☀ **Điểm đặc biệt của LANG SON OJ:** Thí sinh sau khi nộp bài có thể xem lại chi tiết từng testcase đã chấm!

- Bấm vào từng testcase trong danh sách kết quả để xem: **Trạng thái từng test, Thời gian chạy, Bộ nhớ sử dụng.**
- Đối với các testcase ẩn (Hidden Testcase): Sau khi nộp bài, hệ thống cho phép **Mở khóa (Unlock Hidden Test)** để thí sinh xem trực tiếp dữ liệu Input, Output mong muốn và Output thực tế của chương trình mình sinh ra, giúp việc tìm lỗi (debug) cực kỳ nhanh chóng và hiệu quả.

3. Ý NGHĨA CÁC MÃ TRẠNG THÁI CHẤM BÀI (VERDICTS)

Hệ thống sử dụng các mã phán quyết chuẩn quốc tế trong tin học thi đấu:

Mã	Tên đầy đủ	Mô tả chi tiết	Cách khắc phục
AC	Accepted	Chấp nhận! Lời giải hoàn toàn chính xác, vượt qua 100% testcase trong thời gian và bộ nhớ cho phép.	Xuất sắc! Có thể chuyển sang bài tiếp theo hoặc tối ưu thêm mã nguồn.
WA	Wrong Answer	Kết quả sai. Output chương trình in ra không khớp với đáp án chuẩn của đề bài.	Kiểm tra trường hợp biên (corner cases), số lớn (dùng <code>long long</code>), hoặc thừa/thiếu ký tự xuống dòng.
TLE	Time Limit Exceeded	Quá thời gian. Thuật toán chạy vượt quá giới hạn thời gian (ví dụ > 1.0 giây).	Tối ưu độ phức tạp thuật toán (từ $O(N^2)$ xuống $O(N \log N)$), dùng Fast I/O trong C++ (<code>cin.tie(NULL)</code>).
MLE	Memory Limit Exceeded	Quá bộ nhớ. Chương trình cấp phát RAM vượt giới hạn cho phép (ví dụ > 256 MB).	Giảm kích thước mảng tĩnh, tránh tạo quá nhiều đối tượng trong vòng lặp hoặc giải phóng bộ nhớ thừa.
CE	Compile Error	Lỗi biên dịch. Code có lỗi cú pháp, sai tên hàm, thiếu thư viện,... không thể biên dịch thành file chạy.	Đọc bằng báo lỗi compiler hiển thị bên dưới để biết chính xác số dòng bị lỗi cú pháp.
RTE	Runtime Error	Lỗi thực thi. Chương trình bị crash khi đang chạy: chia cho 0, truy cập mảng ngoài phạm vi (out-of-bounds), tràn ngăn xếp đệ quy.	Kiểm tra chỉ số mảng ($A[i]$ khi $i < 0$ hoặc $i \geq \text{MAX}$), dùng đệ quy đúng thời điểm.

4. HƯỚNG DẪN DÀNH CHO QUẢN TRỊ VIÊN (ADMIN)

4.1. Đăng nhập trang quản trị

1. Truy cập đường dẫn `/admin` trên trình duyệt.
2. Nhập **Khóa bí mật Admin (Admin Secret Key)** được cấp khi cài đặt hệ thống.
3. Nhấn **"Đăng nhập quản trị"**. Sau khi xác thực thành công, giao diện quản lý toàn diện sẽ hiển thị.

4.2. Quản lý danh sách bài tập

Tại trang Admin Dashboard, quản trị viên có thể:

- Theo dõi thống kê tổng số bài tập, phân loại theo độ khó.
- Tìm kiếm bài tập cần chỉnh sửa.
- **Chỉnh sửa bài tập (Nút Sửa):** Cập nhật lại nội dung đề bài, thời gian, bộ nhớ, thêm bớt testcase.
- **Xóa bài tập (Nút Xóa):** Xóa bỏ bài tập không còn sử dụng khỏi hệ thống (có hộp thoại xác nhận an toàn).
- **Tải file mẫu & hướng dẫn:** Có các nút bấm trực tiếp để tải `sample.txt` và `guild.txt` về máy tính bất kỳ lúc nào.

4.3. Hai chế độ thêm bài tập mới

Khi nhấn nút **"Thêm bài tập mới"** (`/admin/problems/new`), hệ thống cung cấp **2 chế độ**:

Chế độ 1: Nhập thủ công (Form truyền thống)

Quản trị viên điền trực tiếp thông tin vào các trường biểu mẫu: Mã bài (Slug), Tên bài (Title), Độ khó, Giới hạn thời gian, Giới hạn bộ nhớ, Nội dung đề bài (hỗ trợ xem trước Markdown), và nhấn nút **"Thêm Testcase"** để nhập Input, Output mong muốn cho từng test.

Chế độ 2: Nhập từ file mẫu (Khuyến dùng - Nhanh gấp 10 lần)

Chế độ này cho phép giáo viên soạn sẵn đề bài và hàng chục testcase trong một file văn bản `.txt` trên máy tính (Notepad, VS Code, Word,...), sau đó chỉ cần kéo thả file lên web, hệ thống sẽ **tự động phân tích cú pháp, điền sẵn toàn bộ dữ liệu vào form**.

Quy trình 3 bước tạo bài siêu tốc:

1. Tải file mẫu `sample.txt` trên trang Admin về máy.
2. Mở file, điền nội dung bài và các testcase theo mẫu.
3. Kéo thả file vào ô tải lên → Nhấn **"Tạo bài tập"**. Hoàn tất trong chưa đầy 30 giây!

4.4. Quy chuẩn cú pháp file mẫu (`sample.txt`)

File mẫu được phân chia thành 3 khối thẻ chính rất dễ nhớ:

```
[METADATA]
title: Tên bài toán
slug: ma-bai-toan-viet-lien-khong-dau
difficulty: EASY | MEDIUM | HARD
timeLimit: 1.0
memoryLimit: 256
tags: Quy hoạch động, Mảng

[PROBLEM]
# Mô tả bài toán
Nội dung đề bài chi tiết ở đây, hỗ trợ công thức LaTeX:  $\$1 \leq N \leq 10^5\$$ .

## Định dạng đầu vào
Mô tả dòng vào...

## Định dạng đầu ra
Mô tả kết quả in ra...

## Ràng buộc
-  $\$1 \leq N \leq 10^5\$$ 

[TESTCASE]
isSample: true
explanation: Giải thích test ví dụ nếu có
---INPUT---
5
1 2 3 4 5
---OUTPUT---
15

[TESTCASE]
isSample: false
---INPUT---
3
10 20 30
---OUTPUT---
60
```

Quy tắc cần nhớ khi soạn file:

- `[METADATA]` : Chứa thông tin cấu hình cơ bản. `difficulty` phải là một trong 3 giá trị: `EASY` , `MEDIUM` , `HARD` .
- `[PROBLEM]` : Chứa toàn bộ nội dung đề bài dạng Markdown chuẩn.
- `[TESTCASE]` : Mỗi testcase bắt đầu bằng thẻ này.
 - `isSample: true` đối với các test ví dụ hiển thị công khai ở đề bài.

- `isSample: false` đối với các testcase bí mật dùng để chấm điểm.
- Dữ liệu test được ngăn cách bởi hai mốc rõ ràng: `---INPUT---` và `---OUTPUT---`.

⚠ **Chú ý:** Không xóa hoặc sửa đổi tên các mốc `---INPUT---` và `---OUTPUT---` để bộ phân tích cú pháp đọc dữ liệu chính xác 100%.

5. CÂU HỎI THƯỜNG GẶP & KHẮC PHỤC SỰ CỐ (FAQ)

Q1: Tại sao code của tôi chạy trên máy đúng mà nộp lên hệ thống lại bị Wrong Answer (WA)?

Trả lời: Có một số nguyên nhân phổ biến sau:

- Thừa ký tự khoảng trắng hoặc thiếu ký tự xuống dòng (`\n`).
- Dữ liệu test lớn vượt quá phạm vi số nguyên 32-bit (hãy dùng `long long` trong C++ hoặc số nguyên lớn).
- In thêm các dòng nhắc nhập liệu không có trong đề bài (ví dụ: `cout << "Nhap n: ";` hoặc `input("Nhap n: ")`). Trên OJ, chỉ đọc dữ liệu thô và in đúng định dạng yêu cầu.

Q2: Làm thế nào để tăng tốc độ vào/ra dữ liệu (Fast I/O) tránh bị TLE trong C++?

Trả lời: Trong C++, hãy đặt hai dòng lệnh này ngay đầu hàm `main()` và dùng `'\n'` thay cho `endl` :

```
ios_base::sync_with_stdio(false);
cin.tie(NULL);
```

Q3: Tôi có thể xem lại mã nguồn của các lần nộp trước không?

Trả lời: Hoàn toàn được. Truy cập mục "**Lịch sử nộp bài**" (**Submissions**) trên thanh điều hướng, tìm bài nộp của bạn và nhấn "**Chi tiết**" để xem lại toàn bộ mã nguồn, thời gian nộp và kết quả từng testcase.

Q4: Khi tạo bài từ file .txt bị báo lỗi phân tích, phải xử lý thế nào?

Trả lời: Hãy kiểm tra xem file có đủ 3 khối `[METADATA]` , `[PROBLEM]` , và các khối `[TESTCASE]` hay không. Đồng thời đảm bảo mỗi testcase đều có cặp thẻ phân cách `---INPUT---` và `---OUTPUT---` . Bạn có thể mở file `guild.txt` để đối chiếu lại mẫu chuẩn.

HỆ THỐNG TRỰC TUYẾN LANG SON OJ

Thiết kế & Lập trình bởi: **NgHuyHoang** • Phiên bản 1.0 • Năm 2026